Chris Sun

fzgt320@gmail.com | (+61) 0400 297 972 | github.com/Fzgt | linkedin.com/in/fzgt
Full working rights (Subclass 485) · Sydney, NSW · Open to relocating

TECHNICAL SKILLS

Programming Languages: TypeScript, JavaScript, Python

Frontend: React, Next.js, Vue, Angular, Tailwind, TanStack Query, Zustand, R3F, Three.js

Backend & Data: Node.js, NestJS, FastAPI, Django, PostgreSQL, pgvector, MongoDB, WebSockets

Engineering: Playwright, Jest, Vitest, Webpack, Vite, Jenkins, GitHub Actions, Docker, AWS, Cloudflare

AI: LLM Integration, Streaming Responses, Structured Outputs, Embeddings, Vector Search, RAG

EDUCATION

University of Technology Sydney (UTS) Aug 2024 – Jun 2026
Master of Information Technology, Enterprise Software Development *Sydney, Australia*

Shandong Technology and Business University Sep 2017 – Jun 2021
Bachelor of Science, Information and Computing Science *Shandong, China*

EXPERIENCE

Beyz Jan 2024 – Oct 2024
Full-stack Engineer *Canada, Remote*

– Built the real-time interview copilot end to end: tab and system audio through getDisplayMedia, streamed over a WebSocket to Deepgram and answered by GPT-4o token by token as each utterance ends.
– Minted 60-second usage-scoped Deepgram keys per session behind a plan gate, socket alive across pauses.
– Owned the paid conversion path — Stripe Checkout, trials and lifetime purchases on Next.js and Drizzle — with entitlements written only by signature-verified webhooks: no event can downgrade a lifetime purchase.
– Sole author of a NestJS service for subscription emails and scheduled payment reminders, Dockerized and deployed to EC2 through GitHub Actions, and of the repo's first CI gate on every push.

BioMap Apr 2022 – Oct 2022
Frontend Engineer *Beijing, China*

– Led the React 18 and TypeScript architecture for Digital OS, the team's core consumer-facing product, enforcing feature boundaries with five custom ESLint import zones: no feature can reach into another.
– Built the design system on Radix UI, Storybook and CVA over MSW mocks, variants typed off the styles.
– Layered TanStack Query in three tiers — API functions, queryOptions factories, typed hooks — single-source cache keys, mutation-driven invalidation and hover prefetch: detail pages open instantly, with no request at all.
– Developed the LIMS platform in Vue 3 and TypeScript for sample management and SOP workflows, extending the Luckysheet SDK with custom cell actions and a per-cell JSON contract, plus a scaffolding CLI.

Baidu Jul 2021 – Apr 2022
Frontend Engineer *Beijing, China*

– Built a virtualized React feed serving 5M+ daily page views on a custom useVirtualList hook over IntersectionObserver: live DOM nodes stay near 100 at any scroll depth, scroll from 30 to a stable 60 FPS.
– Led a Webpack 3-to-5 migration across 10+ incompatible packages, extracting vendor and runtime chunks and turning on persistent caching — bundle size down ~15%, local build times down 70% on warm builds.

– First to diagnose a P2 white screen on Haokan Video's search page (30M+ DAU): mismatched Node versions regenerated package-lock.json, pulling a breaking minor bump — version locks and release checks followed.
– Built the withdrawal client on a path peaking above 2,000 QPS: request locking drops in-flight duplicates, client-generated idempotency keys survive reloads, and uncertain outcomes surface as failures, never successes.
– Delivered the Xiaodu App's H5 smart-home module in Vue 2 on Webpack 3, embedded in the native shell: iOS/Android sharing through jsBridge, device detail pages, and event tracking on the Baidu Statistics API.

PROJECTS

WinterFell — WebGPU Neon Runner

React Three Fiber · Three.js · WebGPU · TypeScript

– Built a browser racing game: steer a craft through fields of neon obstacles across twenty scenes, with the terrain, skyline and obstacles all generated on the fly from how far you have traveled, on a fresh seed each run.
– Made the neon glow work on machines without WebGPU: it is built on the newer three.js node pipeline, so one renderer drives WebGPU where the browser has it and falls back to WebGL2 where it does not.
– Cut draw calls from 129 to 76 by collapsing each obstacle field into two instanced meshes, measured frame by frame in a headless Chromium harness: the cost is that off-screen obstacles can no longer be skipped one by one.
– Checked the level generator by machine rather than by playing it: across 1,020 generated layouts, every step tested against how far the craft can move sideways at that stage's speed, every layout had a way through.
– Traced a freeze on every color change to the fog: rebuilding it as a new object made every material in the scene recompile its shader in one frame. Reusing one fog object and setting its color removed it.

Periplus — LLM Research Pipeline

Python · FastAPI · LLM Orchestration · pgvector

– Six-stage LLM pipeline behind typed Pydantic stage contracts filled by schema-constrained JSON output, with run state and stage artifacts in PostgreSQL: a run that dies mid-pipeline resumes from the last completed stage.
– Cached evidence embeddings in pgvector on an ivfflat cosine index, generated locally by MiniLM.
– Put a scripted seam behind every external dependency — the model provider, search, httpx page fetch and the clock — so the whole system runs with no API keys and no network: 461 pytest tests, 1.5 seconds end to end.
– Ran an offline eval harness against a frozen golden set, asserting ranges rather than literal output on evidence-gap rate, citation integrity and token cost: every prompt change lands as a before/after diff.